

1. La Estrategia de la Snapshot (El "Check Point")

Lo que el ingeniero va a hacer:

Probablemente les diga: "Abran su VMware, reviertan a la Snapshot que enviaron/guardaron y ejecuten el código".

Tu preparación:

- Asegúrate de que la Snapshot "oficial" compile y corra a la primera.
 - **Ten preparado el entorno:** Que al revertir, la terminal ya esté visible, el código esté en la carpeta correcta y sepas el comando de compilación de memoria (gcc -o final Final.c). No pierdas tiempo buscando archivos.
-

2. Nivel 1: Preguntas de Comprensión (Teoría sobre el Código)

Antes de pedirles que toquen código, querrá saber si lo entienden o si lo copiaron.

- **Pregunta Clave:** "¿Por qué usan una lista enlazada para los procesos y un arreglo para las particiones?"
 - **Respuesta:** "Usamos un arreglo (tabla_particiones) porque las particiones son estáticas y fijas (sabemos cuántas hay desde el inicio). Usamos una lista enlazada (lista_procesos) porque no sabemos cuántos procesos llegarán ni en qué orden terminarán, así que necesitamos una estructura dinámica que crezca y decrezca."
 - **Pregunta Clave:** "¿Qué representa la variable memoria?"
 - **Respuesta:** "Es un puntero a entero (int *) que usamos con malloc para simular la RAM física. Allí escribimos el ID del proceso para visualizar la ocupación."
 - **Pregunta Clave:** "¿Qué tipo de fragmentación genera su código?"
 - **Respuesta:** "Fragmentación **Interna**, porque asignamos una partición fija completa a un proceso que puede ser más pequeño, desperdiciando el espacio sobrante dentro de la partición."
-

3. Nivel 2: Las Modificaciones en Vivo (La parte difícil)

Basado en lo que contó tu compañero, prepárate para editar el Final.c frente a él. Tienes que ser rápido. Usa nano, vim o el editor de texto de OpenSUSE (como gedit o VS Code si lo tienes instalado en la VM), pero sé rápido.

Aquí están los 3 escenarios más probables de modificación:

Escenario A: "Hagan que un proceso falle por tamaño"

Contexto: En tu código actual (línea 142), tamaño_proceso se calcula con modulo % tamaño_particion, por lo que **NUNCA** es más grande que la partición.

La Petición: "Quiero ver qué pasa si llega un proceso de 200KB a sus particiones de 100KB. Demuéstrelo ahora."

La Solución Rápida:

Ve a la función crear_proceso y modifica la generación del random:

C

```
/* CAMBIO EN VIVO: Sumar 100 para forzar que sea más grande */
```

```
tamano_proceso = (rand() % (tamano_particion + 100)) + 1;
```

Compila y ejecuta. Verás que sale el mensaje de error que ya tienes programado.

Escenario B: "Muestren cuánta memoria total se está desperdiciando"

Contexto: Ya calculas la fragmentación por proceso, pero quizás pida el total global.

La Petición: *"Agreguen al final de la tabla de particiones un resumen del total de KB perdidos por fragmentación interna."*

La Solución Rápida:

Ve a mostrar_tabla_particiones. Agrega una variable acumuladora.

C

```
void mostrar_tabla_particiones(void) {
```

```
    /* ... variables ... */
```

```
    int desperdicio_total = 0; // NUEVA VARIABLE
```

```
    /* ... dentro del for ... */
```

```
    if (tabla_particiones[i].estado == 1) {
```

```
        /* ... lógica existente ... */
```

```
        /* SUMAR AL ACUMULADOR */
```

```
        desperdicio_total += (tabla_particiones[i].tamano - proceso_actual->tamano_requerido);
```

```
    }
```

```
    /* ... al final de la función, antes de cerrar ... */
```

```
    printf("\nTOTAL MEMORIA DESPERDICIADA (Frag. Interna): %d KB\n", desperdicio_total);
```

```
}
```

Escenario C: "Cambien el tamaño de las particiones"

Contexto: Tu código actual pide el tamaño al inicio y hace todas iguales.

La Petición: *"Quiero que la primera mitad de la memoria tenga particiones de 100KB y la segunda mitad de 200KB."*

La Solución Rápida:

Esto es más complejo. Tienes que ir a inicializar_memoria y cambiar el bucle for.

Si te pide esto, respira hondo.

C

```

/* En inicializar_memoria */

/* Reemplaza el cálculo automático de num_particiones y hazlo manual o divídelo */
for (i = 0; i < num_particiones; i++) {
    // ... ids y estados ...

    /* MODIFICACION */
    if (i < num_particiones / 2) {
        tabla_particiones[i].tamano = 100; // Mitad particiones chicas
    } else {
        tabla_particiones[i].tamano = 200; // Mitad particiones grandes
    }

    // IMPORTANTE: Recalcular direccion_inicio basado en los tamaños variables
    // Esto es difícil de hacer en vivo en 2 minutos.
    // MEJOR RESPUESTA: "Ingeniero, como calculamos la dirección de inicio
    // secuencialmente, cambiar los tamaños requiere reestructurar el bucle
    // de direcciones. ¿Prefiere que modifique solo el tamaño lógico?"
}

```

4. Checklist para el Día D (3 de Febrero)

Como son 2 personas, dividan roles, pero ambos deben saber hacer todo:

1. **El "Piloto":** Quien maneja el teclado. Debe saber moverse rápido en Linux (comandos ls, gcc, ./, clear).
2. **El "Copiloto":** Quien habla con el ingeniero. Si el Piloto se traba programando, el Copiloto explica: *"Lo que mi compañero está haciendo es modificar la variable random para..."*. Esto rellena el silencio incómodo.

Comandos que deben tener en la punta de los dedos:

- Compilar: gcc -o test Final.c (Ojo: Si usas math.h en el futuro, recuerda -lm, pero aquí no hace falta).
- Ejecutar: ./test
- Listar archivos: ls -l
- Limpiar pantalla: clear

Resumen de Predicción

Es muy probable que el ingeniero quiera ver:

1. Que el programa **no trueque** (Segmentation Fault).
 2. Que manejes el caso de **Memoria Llena**. (Crea procesos hasta llenar todo. ¿Qué pasa? ¿Muestra mensaje o se cierra?).
 3. Que manejes el caso de **Proceso Gigante** (Modificando el random).
-

PARTE 1: Explicación "Flash" del Código (Para decir en 1 minuto)

Olvidate de explicar línea por línea. Usa esta analogía y estructura. Si te preguntan "¿Cómo funciona?", di esto:

1. El Concepto (La Analogía)

"Ingeniero, nuestro programa simula un **estacionamiento privado (La Memoria)**.

- Las **Particiones** son los cajones de estacionamiento: Son fijos, hay una cantidad exacta y no cambian de tamaño.
- Los **Procesos** son los autos: Llegan, buscan el *primer* lugar donde quepan (Primer Ajuste), se estacionan un tiempo y luego se van."

2. Las Estructuras (¿Por qué usaron esto?)

- **Particion (Array)**: "Usamos un arreglo (tabla_particiones) porque sabemos el número exacto de particiones desde el inicio. Es estático."
- **Proceso (Lista Enlazada)**: "Usamos una lista enlazada (lista_procesos) porque no sabemos cuántos procesos van a llegar ni en qué orden se irán. Es dinámica."
- **int *memoria**: "Es solo un vector visual. Lo llenamos con el ID del proceso para que el usuario 'vea' la memoria ocupada, pero la lógica real está en la tabla de particiones."

3. El Algoritmo (Primer Ajuste)

"Recorremos el arreglo de particiones con un for. La primera que esté Libre (Estado 0) y cuyo tamaño sea suficiente, esa tomamos. Y usamos break para dejar de buscar."

PARTE 2: Los 3 Cambios Realistas (Live Coding)

El ingeniero querrá que cambies una línea para ver un resultado inmediato. Ten el archivo abierto en el editor (nano, gedit, vim) y ubica estas zonas.

Escenario A: "Haz que el proceso sea más grande que la partición (Forzar Error)"

Objetivo: Demostrar que tu validación de tamaño funciona.

Tiempo: 30 segundos.

Dónde: Función crear_proceso(), busca donde se calcula tamano_proceso.

Acción: Súmale algo al random.

C

/ CODIGO ORIGINAL */*

tamano_proceso = (rand() % tamano_particion) + 1;

```
/* CAMBIO EN VIVO (Sumar 100 asegura que sea mayor que la partición) */
```

```
tamano_proceso = (rand() % tamano_particion) + 100;
```

Resultado: Al ejecutar y crear proceso, saldrá tu mensaje: "Error: No hay partición con espacio suficiente".

Escenario B: "Muestra el ID real del proceso en la visualización de memoria"

Objetivo: Ver si sabes cómo se imprime la memoria.

Tiempo: 45 segundos.

Dónde: Función mostrar_memoria().

Acción: Cambiar el carácter visual por el número.

C

```
/* CODIGO ORIGINAL (Muestra bloques ■ o similar) */
```

```
printf("[ ] "); // O el caracter que tengas
```

```
/* CAMBIO EN VIVO */
```

```
if (memoria[i] == -1)
```

```
    printf("[Libre] ");
```

```
else
```

```
    printf("[%d] ", memoria[i]); // Muestra el ID numérico
```

Escenario C: "Haz que fallen procesos por Memoria Llena RÁPIDO"

Objetivo: No estar creando 50 procesos uno por uno. Quieren llenar la memoria en 3 clics.

Tiempo: 30 segundos.

Dónde: Función inicializar_memoria().

Acción: Reduce drásticamente el tamaño de la memoria total.

- Al correr el programa, cuando te pida "Ingrese tamaño total de memoria":
 - Ponle **200** (en vez de 1000).
 - Ponle tamaño de partición **100**.
 - *Resultado:* Solo se crean 2 particiones. Creas 2 procesos y al tercero saldrá "Memoria Llena". ¡Listo!

PARTE 3: Preguntas "Trampa" (Respuestas cortas)

1. Ing: "¿Por qué hay fragmentación interna?"

- **Tú:** "Porque las particiones son fijas. Si la partición es de 100KB y el proceso usa 1KB, desperdiciamos 99KB que nadie más puede usar."

2. Ing: "Cámbiame el algoritmo a Mejor Ajuste (Best Fit) ahorita."

- **Tú (Respuesta Salvavidas):** "Ingeniero, como el enunciado pedía particiones de **igual tamaño**, el 'Mejor Ajuste' funciona matemáticamente igual al 'Primer Ajuste' (todas las particiones libres son iguales). Para que funcione diferente, tendría que cambiar la inicialización para tener particiones de diferentes tamaños. ¿Quiere que haga eso?"
 - *(90% de probabilidad de que diga "Ah, cierto, déjalo así").*

3. Ing: "¿Qué pasa si libero memoria del puntero memoria pero no del nodo de la lista?"

- **Tú:** "Tendríamos un **Memory Leak** (fuga de memoria). El proceso seguiría existiendo en la lista lógica, pero sin espacio físico asignado."
-

Tu Guía Paso a Paso para la Defensa (Checklist)

1. **Inicio:** Abran la Snapshot. Compilen (gcc ...). Ejecuten.
2. **Muestra:** Creen 2 procesos. Muestren la tabla. Cierren 1. Muestren que se liberó.
3. **El Reto:** Esperen la pregunta.
 - Si pide código: Usen el **Escenario A** (es el más fácil y visual).
 - Si pide teoría: Usen la **Analogía del Estacionamiento**.
4. **Cierre:** Muestren el resumen de estadísticas si lo tienen y listo.